

Modelos de Otimização para Alocação de Recursos Humanos em Projetos de Software

Daniel Arruda de Santana
Departamento de Informática
Universidade Estadual de Maringá – UEM
Maringá, Brasil
<https://orcid.org/0009-0005-3165-8272>

Luciano Trevisan Alberti
Departamento de Informática
Universidade Estadual de Maringá – UEM
Maringá, Brasil
<https://orcid.org/0009-0006-1557-6602>

Resumo—Este trabalho investiga o uso de metaheurísticas na alocação de recursos humanos em projetos de software, problema combinatório de natureza NP-difícil que envolve competências, custos, disponibilidades e prazos conflitantes. São apresentadas as formulações mais recorrentes na literatura, com destaque para as funções objetivo adotadas e para o tratamento das restrições de capacidade e precedência entre tarefas. As principais técnicas são comparadas quanto à qualidade das soluções obtidas, ao custo computacional e à capacidade de lidar com objetivos conflitantes, com ênfase em algoritmos genéticos e abordagens multiobjetivo. Discute-se ainda a distinção entre alocação estática e dinâmica, evidenciando que a segunda impõe requisitos adicionais de reprogramação em tempo de execução ainda pouco explorados. Os resultados indicam que abordagens multiobjetivo oferecem maior flexibilidade ao gestor por fornecerem um conjunto de soluções de compromisso, embora ao custo de maior esforço de parametrização.

Palavras-chave—Alocação de recursos humanos, Algoritmos genéticos, Engenharia de software baseada em busca, Meta-heurísticas, Otimização multiobjetivo, Projetos de software

I. PROBLEMÁTICA

A alocação de profissionais entre projetos e tarefas constitui um problema relevante de gestão de recursos em Engenharia de Software, especialmente em cenários nos quais existem múltiplos projetos concorrentes, diferentes competências profissionais, restrições de disponibilidade, custos associados aos recursos humanos e prazos de execução. A combinação dessas variáveis torna o processo de alocação uma atividade complexa, na qual uma decisão que favorece determinado projeto pode produzir impactos negativos em outros.

Considere-se, como cenário hipotético representado na Tabela I, uma organização composta por **10 profissionais, responsáveis pela execução de 5 projetos contendo, ao todo, 30 tarefas**. Cada profissional apresenta **diferentes níveis de competência, custos de alocação e períodos de disponibilidade, enquanto as tarefas possuem diferentes requisitos de competência e prazos de execução**. Acrescentando ao exemplo hipotético, pode-se incrementar ao cenário as seguintes condições:

A. Variáveis de decisão

- Qual profissional será atribuído a cada tarefa?
- Em qual projeto?
- Em qual período?

B. Restrições

- Disponibilidade do profissional;
- Competências necessárias;
- Capacidade máxima;
- Prazo das tarefas;
- Possibilidade de um profissional atuar em múltiplas tarefas/projetos.

C. Função objetivo

- Minimizar custo;
- Minimizar atrasos;
- Maximizar aderência entre competência e tarefa;
- Maximizar utilização dos profissionais;
- Ou uma combinação ponderada desses objetivos.

Tabela I
REPRESENTAÇÃO SIMPLIFICADA DO PROBLEMA

Elemento	Cenário
Profissionais	10
Projetos	5
Tarefas	30
Competências	Diferentes entre profissionais e tarefas
Custo	Diferente por profissional
Disponibilidade	Diferente por profissional
Prazo	Diferente por projeto/tarefa

Nesse contexto, o problema consiste em determinar como distribuir os profissionais entre os projetos e tarefas de modo a obter uma configuração de alocação que atenda às restrições existentes e, simultaneamente, otimize os critérios de custo, prazo, utilização da capacidade e adequação das competências.

A complexidade do problema aumenta quando se considera a diferença entre alocação estática, na qual os profissionais são atribuídos previamente às atividades, e alocação dinâmica, na qual as atribuições podem ser modificadas ao longo da execução dos projetos em função de alterações de disponibilidade, prioridades, atrasos ou novas demandas.

Diante desse cenário, formula-se a seguinte questão:

Quais técnicas computacionais podem ser utilizadas para otimizar a alocação de profissionais entre projetos e tarefas, considerando simultaneamente restrições de custo, prazo, competências e capacidade? De forma mais categórica: **Qual configuração otimiza a função objetivo?**

II. JUSTIFICATIVA

O problema de alocação de profissionais, como o apontado e exemplificado anteriormente, apresenta característica combinatória, uma vez que o número de possíveis configurações cresce significativamente à medida que aumentam a quantidade de profissionais, tarefas e projetos. Para cada tarefa, diferentes profissionais podem ser considerados candidatos à execução, e cada nova atribuição pode alterar a disponibilidade dos recursos, os prazos dos projetos e o custo total da solução.

Considerando, de forma simplificada, um cenário com 10 profissionais e 30 tarefas, caso cada tarefa pudesse ser atribuída a qualquer profissional, existiriam até 10^{30} combinações possíveis de alocação. Embora o problema real envolva diversas restrições que reduzam o conjunto de soluções viáveis, o exemplo evidencia a rápida expansão do espaço de busca.

A inclusão de outras características, como diferentes competências, disponibilidades, custos, prioridades e dependências entre tarefas, aumenta ainda mais a complexidade do problema. Além disso, quando se considera a possibilidade de realocar profissionais durante a execução dos projetos, o problema passa a envolver também decisões ao longo do tempo, ampliando o espaço de soluções a ser explorado.

Nesse contexto, uma abordagem baseada na avaliação exaustiva de todas as configurações possíveis pode se tornar computacionalmente inviável à medida que o tamanho do problema aumenta. Métodos exatos podem encontrar uma solução ótima em determinadas formulações, porém o custo computacional associado à exploração do espaço de soluções pode crescer de forma significativa.

As metaheurísticas apresentam-se, portanto, como uma alternativa para esse tipo de problema. Em vez de explorar sistematicamente todas as possibilidades, essas técnicas empregam estratégias de busca capazes de explorar grandes espaços de soluções e direcionar o processo para regiões potencialmente mais promissoras. Dessa forma, busca-se obter soluções de alta qualidade em tempo computacional aceitável, mesmo quando a obtenção da solução ótima por métodos exatos se torna impraticável.

Assim, a utilização de metaheurísticas neste problema é justificada pela combinação entre um grande espaço de soluções, múltiplas restrições e diferentes objetivos de otimização. A abordagem permite investigar estratégias de alocação capazes de equilibrar fatores como custo, cumprimento de prazos, adequação das competências e utilização da capacidade dos profissionais.

III. POSSÍVEIS SOLUÇÕES

Para exemplificar possíveis soluções, houve o levantamento sobre três artigos e que são tratados nas subseções seguintes, tendo como elemento central o **Algoritmo Genético**, onde ele é apenas o mecanismo e as diferenças passam a existir na modelagem do problema.

A. Uso de Algoritmo Evolutivo Cooperativo

Para solucionar o problema de alocação de pessoas em projetos de software, [1] propõem o uso de um Algoritmo Evolutivo Cooperativo (*Cooperative Co-evolutionary Algorithm*, CCEA). O ponto central da abordagem é dividir o problema em dois subproblemas inter-relacionados:

- 1) A alocação de pessoas (staff) em equipes;
- 2) O agendamento das tarefas (work packages) para essas equipes.

Cada subproblema é tratado como uma população distinta dentro do algoritmo evolutivo, permitindo que soluções parciais para cada aspecto evoluam de forma independente, mas cooperem para formar uma solução global melhor.

A representação das soluções é feita por cromossomos: um cromossomo representa a distribuição dos *work packages* entre as equipes (ordenação das tarefas), enquanto outro cromossomo representa a alocação dos membros do *staff* nas equipes, conforme ilustrado na Figura 1. O algoritmo evolui essas duas populações em paralelo, avaliando a qualidade das soluções com base no tempo total de conclusão do projeto. Durante o processo evolutivo, operadores genéticos como **crossover** e **mutação** são aplicados para gerar novas soluções, e há mecanismos para garantir que todas as equipes tenham pelo menos um membro e que as dependências entre tarefas sejam respeitadas.

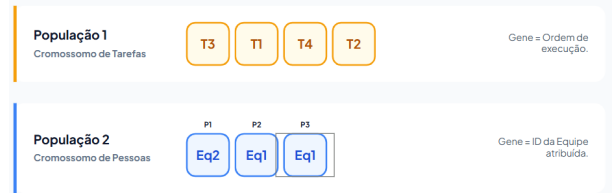


Fig. 1. Representação do CCEA

A cooperação entre as populações ocorre porque a avaliação de uma solução em uma população depende da melhor solução encontrada na outra população. **Por exemplo**, para avaliar uma determinada alocação de pessoas, utiliza-se a melhor ordem de tarefas encontrada até então, e vice-versa (exemplificado na Figura 2). Isso permite que as soluções evoluam de forma "simbiótica", buscando sempre melhorar o resultado global do projeto.



Fig. 2. A avaliação simbiótica

No trabalho, o experimento foi realizado sobre quatro projetos reais e com uma pergunta: **Quanto as duas populações devem conversar?**. A Figura 3 mostra como essa abordagem

foi projetada e também ilustra as três configurações testadas para as suposições de comunicação entre as populações.

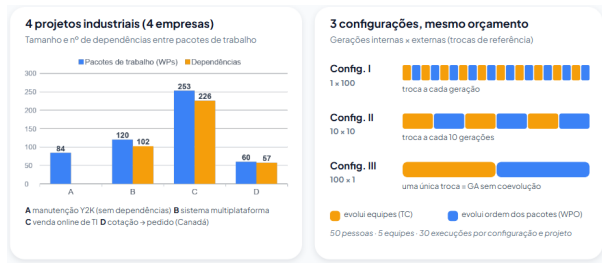


Fig. 3. O Experimento Realizado

Os autores demonstraram empiricamente que essa abordagem cooperativa supera tanto algoritmos evolutivos tradicionais de população única quanto buscas aleatórias, especialmente em projetos com dependências complexas entre tarefas. Conforme mostrado na Figura 4, o método mostrou-se mais eficiente e eficaz na redução do tempo total de conclusão dos projetos analisados.

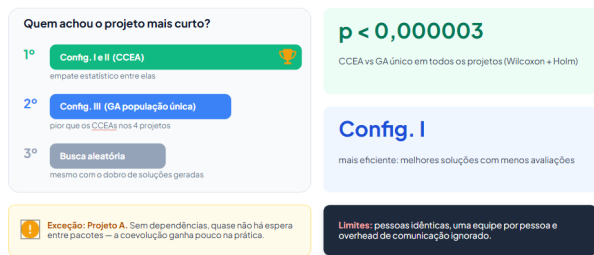


Fig. 4. Conclusão

Há uma lacuna neste trabalho em que os autores fazem a seguinte suposição para simplificar a abordagem: “os membros da equipe são idênticos em termos de habilidades e expertise, e cada membro trabalha apenas em uma equipe durante todo o projeto”. Além dessa limitação mencionam que, para trabalhos futuros, pretendem considerar modelos mais sofisticados que levem em conta fatores como habilidades e expertise dos desenvolvedores, além de outros aspectos como overhead de comunicação e robustez do cronograma.

B. Uso de Algoritmo Genético Multiobjetivo

Para solucionar o problema de alocação de pessoas em projetos de software, [2] propõem o uso de um algoritmo genético multiobjetivo, especificamente o NSGA-II (*Non-dominated Sorting Genetic Algorithm II*). O objetivo é otimizar simultaneamente diferentes aspectos do planejamento de projetos e da formação de equipes, levando em conta critérios como duração do projeto, experiência dos desenvolvedores e tamanho das equipes. A abordagem funciona da seguinte forma: cada solução candidata representa um possível cronograma de tarefas e uma alocação de desenvolvedores para essas tarefas. O algoritmo avalia cada solução com base em três funções-objetivo principais:

- 1) **Duração do projeto:** busca minimizar o tempo total necessário para concluir todas as tarefas, evitando atrasos desnecessários entre atividades dependentes;
- 2) **Adequação de habilidades:** procura maximizar o nível de experiência dos desenvolvedores alocados a cada tarefa, garantindo que as pessoas certas estejam nas tarefas certas, de acordo com as habilidades requeridas;
- 3) **Tamanho da equipe:** incentiva a formação de equipes enxutas, minimizando o número de desenvolvedores por tarefa para reduzir esforços e custos de comunicação.

Além disso, o algoritmo considera restrições importantes, como **dependências entre tarefas** (uma tarefa não pode começar antes que suas predecessoras terminem), **conflitos de alocação** (um desenvolvedor não pode estar em mais de uma tarefa ao mesmo tempo) e a necessidade de que todas as **habilidades exigidas** por uma tarefa estejam presentes na equipe alocada.



Fig. 5. Modelagem do problema

O processo evolutivo do algoritmo gera diversas soluções possíveis, permitindo ao gerente de projetos escolher aquela que melhor equilibra os objetivos conflitantes, como menor duração versus maior experiência da equipe. Os experimentos realizados mostram que a abordagem é eficaz para projetos de pequeno e médio porte, conseguindo gerar soluções viáveis e próximas do ótimo, mesmo diante da complexidade e das restrições típicas do ambiente de desenvolvimento de software.

Sobre a modelagem, cada indivíduo da população representa uma solução candidata para o problema integrado de escalonamento de tarefas e alocação de desenvolvedores. A representação adotada combina, para cada tarefa do projeto, uma variável inteira correspondente ao seu dia de início e um conjunto de variáveis binárias que representam a alocação dos desenvolvedores, sendo o valor 1 utilizado para indicar que determinado desenvolvedor está associado à tarefa e 0 caso contrário. Dessa forma, o cromossomo representa simultaneamente as decisões de quando cada tarefa será iniciada e quais desenvolvedores serão responsáveis por sua execução. A representação está sujeita às restrições do problema, incluindo a disponibilidade dos desenvolvedores, as dependências entre tarefas e o atendimento das competências requeridas pelas tarefas (Figura 6).



Fig. 6. Cromossomo Misto

A utilização de uma abordagem multiobjetivo é necessária porque os objetivos do problema não possuem, em geral, uma solução simultaneamente ótima em todas as dimensões. Por exemplo, reduzir a duração do projeto pode exigir a utilização de desenvolvedores menos adequados às competências das tarefas ou provocar conflitos de alocação; inversamente, priorizar desenvolvedores com maior experiência pode aumentar a duração do projeto. Dessa forma, o algoritmo não busca uma única solução ótima, mas um conjunto de soluções não dominadas, nas quais nenhuma solução pode melhorar um objetivo sem produzir uma degradação em pelo menos outro objetivo. Esse conjunto constitui a fronteira de Pareto. No NSGA-II, a fast non-dominated sort é utilizada para identificar essas soluções não dominadas. Ao final das execuções, os autores combinam as populações finais das 15 execuções de cada cenário e aplicam novamente a ordenação por não-dominância para obter uma fronteira de Pareto combinada, representando as melhores alternativas encontradas para aquele cenário (Figura 7).



Fig. 7. Não existe "O Melhor"

No experimento realizado, essa característica fica evidente: no cenário 1, por exemplo, a combinação das 15 execuções produziu 18 soluções na fronteira de Pareto final. Já em cenários mais complexos, o número de soluções não dominadas pode aumentar substancialmente; no cenário 4, foram identificadas 215 soluções únicas na fronteira de Pareto combinada.

Para a evolução da população, o cruzamento e a mutação são aplicados de forma diferenciada às duas partes da representação cromossômica. Para as variáveis inteiras que representam o dia de início das tarefas, é utilizado o Simulated Binary Crossover (SBX), enquanto as variáveis binárias que representam a alocação dos desenvolvedores são recombinadas por crossover de ponto único (single-point crossover). A probabilidade de recombinação atribuída a cada tarefa é de 0,90. Na mutação, as variáveis inteiras referentes aos dias de início são

modificadas por mutação polinomial, enquanto, nas variáveis binárias de alocação, um bit é selecionado aleatoriamente e seu valor é invertido, isto é, 0 passa a 1 ou 1 passa a 0. A probabilidade de mutação aplicada às variáveis associadas às tarefas é definida como $2/N$, em que N corresponde ao número de tarefas do projeto. Dessa forma, os operadores genéticos preservam a natureza heterogênea da representação, utilizando mecanismos específicos para recombinar e modificar as informações de cronograma e de alocação de recursos humanos (Figura 8).



Fig. 8. Cruzamento e Mutação

Em suma, este estudo propõe uma abordagem baseada no algoritmo genético multiobjetivo NSGA-II para otimizar o cronograma de projetos de software e a alocação de equipes, tratando o gerenciamento de recursos humanos como um problema complexo de otimização. A metodologia desenvolvida busca equilibrar objetivos concorrentes — especificamente a duração do projeto, a adequação das habilidades dos desenvolvedores e a minimização do tamanho da equipe — ao mesmo tempo em que lida estritamente com restrições operacionais, como dependências entre tarefas e conflitos de disponibilidade.

Como mostrado na Figura 9 a avaliação da ferramenta, fundamentada em cenários do mundo real e comparada ao método simulated annealing (AMOS), revelou que a abordagem genética apresenta um desempenho amplamente superior, sendo eficaz na geração de soluções viáveis e ótimas (ou quase ótimas) para projetos de pequeno e médio porte. Observou-se, contudo, que a capacidade do algoritmo sofre limitações de escalabilidade; à medida que o tamanho e a complexidade do projeto aumentam, o modelo encontra dificuldades para satisfazer as restrições operacionais, por vezes gerando cronogramas com intervalos de ociosidade ("gaps") desnecessários entre as tarefas. Apesar desses desafios de dimensionamento, a pesquisa evidencia que o modelo possui valor prático significativo para as organizações, atuando como um sistema inteligente de suporte à decisão que fornece aos gerentes de projeto um portfólio robusto de opções de planejamento baseadas em compensações equilibradas (trade-offs).

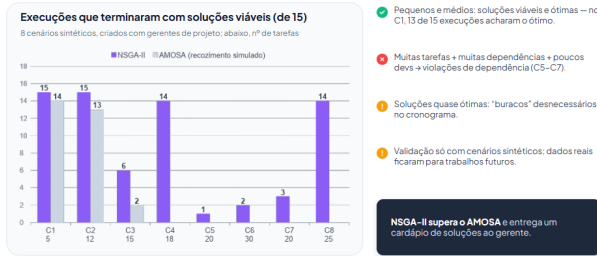


Fig. 9. Conclusão

C. Abordagem Baseada em Algoritmo Genético e Modelo de Tarefa Estruturada (STM)

Para responder à questão formulada e mitigar a complexidade da formação de múltiplas equipes (MTF), especialmente no *framework* ágil *Scrum*, onde o conhecimento costuma ser tácito e as tarefas são de curta duração, a abordagem proposta por [3] estrutura-se em duas etapas fundamentais, ilustrada na Figura 10 e explicadas logo na sequência.



Fig. 10. Abordagem

1. O Modelo de Tarefa Estruturada (STM) A granularidade das tarefas Scrum (geralmente de um dia ou menos) dificulta que algoritmos identifiquem competências reais se descritas apenas em linguagem natural. O STM soluciona isso por meio de dois processos executados pelo time durante o planejamento ou execução das sprints:

- **Processo de Etiquetagem de Tarefas (TLP):** Consiste na atribuição de tags tecnológicas padronizadas à tarefa. Essas etiquetas identificam a camada arquitetural (frontend, backend), linguagem de programação, frameworks, APIs e o banco de dados utilizado. Esse processo quantifica o conhecimento (knowledge) técnico do desenvolvedor;
- **Processo de Categorização de Tarefas (TCP):** Mapeia tarefas que possuem o mesmo propósito prático sob uma descrição padrão (ex: "desenvolver tela de login"). Esse processo permite quantificar as habilidades (skills) práticas adquiridas pelo profissional através da repetição e histórico de implementação.

2. Formação de Equipes via Algoritmo Genético (GA) A compatibilidade técnica entre cada desenvolvedor disponível e cada projeto-alvo é calculada e mapeada em uma matriz. O GA utiliza uma representação de cromossomo de permutação para buscar a melhor combinação global de membros para as equipes e realizar a alocação otimizada. Para realizar a função de *fitness* são usadas duas funções configuráveis:

- **Alocação por Redundância (f1(X)):** Busca formar equipes multifuncionais com sobreposição de conhecimentos es-

pecializados, permitindo que os membros colaborem e se substituam se necessário;

- **Alocação Especializada (f2(X)):** Permite ao gerente de projetos aplicar pesos maiores a tecnologias específicas críticas para o projeto, sem perder a visão de multifuncionalidade.

Na abordagem proposta, a montagem dos cromossomos é concebida através de uma representação de permutação, a qual permite modelar as soluções candidatas por meio da combinação de elementos. Especificamente, a estrutura do cromossomo é definida de modo que seus índices representem as vagas que necessitam ser preenchidas nos projetos da organização. Cada índice corresponde a um gene que assume um valor numérico inteiro, o qual atua como o identificador único de um desenvolvedor disponível para alocação. Consequentemente, o agrupamento sequencial desses genes forma o fenótipo da solução, definindo de maneira direta a alocação de quais membros específicos comporão as equipes de cada projeto alvo (Figura 11).



Fig. 11. Cromossomo de Permutação

No que diz respeito às restrições do modelo proposto, o problema de alocação de equipes é tratado de forma intrínseca por meio da própria representação estrutural das soluções. Ao adotar um cromossomo baseado em permutação, o algoritmo garante que os genes, correspondentes aos identificadores únicos dos desenvolvedores, não se repitam em uma mesma configuração, assegurando que um desenvolvedor não seja alocado em duas equipes simultaneamente, o que respeita a restrição de capacidade limitante e de compartilhamento de recursos humanos dentro da organização. Adicionalmente, o estudo indica que restrições operacionais podem ser atreladas diretamente às funções de aptidão, como a imposição de limites de alocação, onde apenas desenvolvedores que alcancem uma pontuação mínima de competência seriam designados a determinadas vagas.

O processo evolutivo do algoritmo é guiado por mecanismos robustos de seleção e variação que visam otimizar a composição das equipes ao longo das gerações. A seleção de pais para reprodução utiliza o método de torneio, enquanto a preservação das melhores soluções candidatas é resguardada por meio de uma estratégia de seleção de sobreviventes com elitismo. A geração de novos indivíduos ocorre pela recombinação genética via cruzamento parcialmente mapeado (Partially Mapped Crossover), configurado com uma taxa de 50%, e a diversidade genética da população é mantida por meio de um operador de mutação por troca (Swap Mutator) estipulado em 5%. O critério de avaliação da população (*fitness*) determina o nível de alinhamento das equipes pro-

postas por meio de duas abordagens distintas: a "Alocação de Redundância", que busca maximizar a média das competências técnicas para consolidar equipes ágeis multifuncionais com sobreposição de conhecimentos; e a "Alocação Especializada", que emprega fatores de multiplicação para priorizar competências tecnológicas específicas, ajustando os pesos conforme a necessidade delineada pelos gerentes do projeto (Figura 12).



Fig. 12. Avaliação e Geração

A solução foi validada com dados reais de uma organização, mostrando alta precisão (85%) na formação de equipes semelhantes às escolhidas por gerentes de projetos experientes, além de boa aceitação das recomendações (até 75%). Assim, a abordagem automatiza e aprimora a tomada de decisão na alocação de pessoas, tornando o processo mais assertivo, transparente e alinhado às demandas técnicas dos projetos Scrum. Como perspectivas para trabalhos futuros, os autores buscam mitigar limitações iniciais, como o problema de cold start (início a frio) em organizações sem histórico de projetos, sugerindo a integração de fontes de conhecimento externo — a exemplo de redes sociais, repositórios de código e comunidades online — bem como a incorporação de atributos interpessoais e traços de personalidade para enriquecer os perfis dos desenvolvedores.



Fig. 13. Resultados

D. Usando Algoritmos Genéticos Cruzados

Para solucionar o problema de alocação de pessoas em equipes de software, [4] propõem um algoritmo genético (GA) inovador que integra tanto habilidades técnicas quanto colaborativas dos desenvolvedores. O ponto central da abordagem é a utilização de uma função de *fitness* baseada em grafo de colaboração, que avalia não apenas a compatibilidade técnica dos membros da equipe com as demandas do projeto, mas também o histórico de colaboração entre eles (ilustrado na Figura 14). Isso é feito por meio de dados históricos de projetos anteriores, evitando avaliações subjetivas e caras, como testes psicométricos.



Fig. 14. Grafo de colaboração

O algoritmo utiliza uma técnica de cruzamento chamada *Convex Combination Crossover*, que permite uma recombinação mais diversificada das soluções parentais, evitando a convergência prematura e aumentando a diversidade das equipes geradas. O processo de formação de equipes é realizado em etapas:

- Modelagem do problema (incluindo restrições como cobertura de habilidades, tamanho da equipe e histórico de colaboração);
- Quantificação das habilidades técnicas e colaborativas;
- Aplicação do algoritmo genético com operadores de seleção;
- Cruzamento e mutação; e
- Validação das soluções geradas.

No contexto do modelo computacional proposto, a montagem dos cromossomos (Figura 15) atua como a representação estrutural das configurações das equipes de desenvolvimento de software. O algoritmo genético inicia-se com uma população na qual cada cromossomo é formado por genes que correspondem a desenvolvedores individuais. A estruturação de novos cromossomos ao longo das gerações ocorre fundamentalmente pela recombinação de genitores utilizando o operador de cruzamento por combinação convexa (*Convex Combination Crossover*), que mescla os genes de duas soluções parentais. Para garantir que a configuração da equipe gerada seja viável na prática, a montagem de cada cromossomo passa por um processo de validação que assegura a unicidade (corrigindo desenvolvedores duplicados via mutação) e a cobertura completa das habilidades exigidas pelo projeto (substituindo genes conforme a necessidade técnica). Por fim, cada cromossomo recém-montado tem sua viabilidade e qualidade avaliadas por uma função de aptidão (F_{total}) que pondera simultaneamente o alinhamento técnico e o histórico colaborativo da equipe.



Fig. 15. Cromossomos

O processo evolutivo é conduzido pela interação entre os operadores de eleição, combinação, cruzamento e mutação. Inicialmente, a eleição dos indivíduos é realizada por meio do método de torneio (Tournament Selection), que avalia e identifica os cromossomos candidatos com melhor aptidão para avançar à fase de reprodução. A partir dos progenitores eleitos, o cruzamento ocorre mediante a aplicação do operador de Cruzamento por Combinação Convexa (Convex Combination Crossover). Esta etapa de combinação mescla matematicamente os genes (desenvolvedores) de dois parentais com base em um fator de ponderação α (onde $\alpha \in [0, 1]$), gerando assim duas novas configurações de descendentes. Como essa mescla pode gerar soluções estruturalmente inviáveis, a mutação é acionada intrinsecamente durante o cruzamento para substituir desenvolvedores duplicados e garantir a unicidade de cada membro na equipe. Adicionalmente, um operador de mutação padrão é aplicado à população com uma taxa de probabilidade de 0,005, alterando genes pontuais para introduzir diversidade, cobrir habilidades técnicas faltantes e evitar a estagnação prematura do modelo (Figura 16).

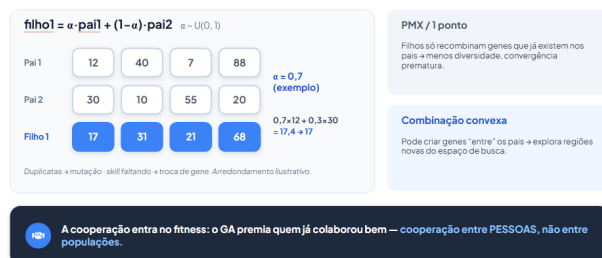


Fig. 16. Cruzamento por Combinação Convexa

O *framework* proposto automatiza a coleta e análise de dados de múltiplas fontes (sistemas de controle de versão, ferramentas de gestão de projetos, feedback de clientes), constrói

perfis detalhados dos desenvolvedores e sugere composições de equipes que maximizam tanto a competência técnica quanto a sinergia colaborativa. Os resultados experimentais mostram que a abordagem supera métodos tradicionais, formando equipes mais equilibradas e eficazes, com maior potencial de sucesso em projetos industriais de software.

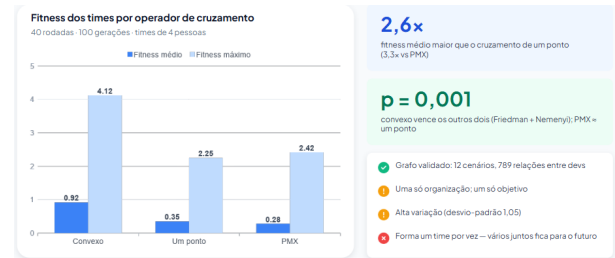


Fig. 17. Resultados

IV. CONSIDERAÇÕES FINAIS

A Figura 18 tem como objetivo realizar uma comparação visual a respeito das particularidades de cada trabalho apresentado anteriormente. Também, na sequência, é feita uma consideração à respeito de todos eles e suas particularidades.

	2011 - Ren	2013 - Stylianou	2022 - Costa	2025 - Cunha
1 O que é uma solução?	Ordem dos pacotes + equipe de cada pessoa	Cronograma + equipe de cada tarefa	Todos os times de todos os projetos	Um time para um projeto
2 Como representar?	2 populações: ordem dos pacotes + pessoa + equipe	1 população: dia de início + bits	Permutação: vaga + dev	Lista de devs do time
3 Como avaliar?	Tempo de conclusão simulado com a outra população	3 objetivos via Pareto (NSGA-II)	Competência média (FI) ou ponderada (F2)	Técnica + colaboração (gráfico)
4 Como gerar?	1 ponto + troca de posição	SBX + 1 ponto + polinomial + bit	PMX + troca (swap)	Combinação convexa + injeção
✓ Validação	4 projetos industriais	8 cenários sintéticos	12 projetos, 4 gerentes: 85%	47 projetos, 149 devs

Fig. 18. Mesmo mecanismo evolutivo, modelagens distintas

A alocação de recursos humanos em projetos de software constitui um problema de otimização de elevada complexidade, especialmente quando são consideradas simultaneamente restrições de disponibilidade, competências, capacidade, dependências entre tarefas, custos e prazos. A análise realizada evidencia que abordagens baseadas exclusivamente na definição manual das equipes podem se tornar pouco adequadas à medida que aumenta o número de profissionais, projetos e tarefas, devido à expansão do espaço de soluções possíveis.

Nesse contexto, as metaheurísticas apresentam-se como uma alternativa para explorar grandes espaços de busca em tempo computacional aceitável, sem a necessidade de avaliar exaustivamente todas as configurações possíveis. Entretanto, a análise das abordagens estudadas demonstra que não existe uma única formulação capaz de representar adequadamente todos os cenários de alocação. A escolha da técnica está diretamente relacionada às características do problema, aos objetivos estabelecidos e às restrições que precisam ser consideradas.

Entre as abordagens analisadas, os algoritmos genéticos apresentam-se como uma alternativa flexível, uma vez que

permitem representar diferentes configurações de equipes e utilizar funções de avaliação adaptadas aos objetivos do problema. A utilização do NSGA-II, por exemplo, permite tratar simultaneamente objetivos conflitantes, como redução da duração do projeto, adequação das competências e redução do tamanho das equipes, oferecendo ao gestor diferentes soluções de compromisso em vez de uma única solução [2]. Essa característica é particularmente relevante em situações nas quais não existe uma configuração que seja simultaneamente ótima em todos os critérios.

Por outro lado, a abordagem baseada em algoritmo evolutivo cooperativo apresenta uma estratégia interessante ao decompor o problema em subproblemas de alocação de profissionais e agendamento de tarefas [1]. Essa decomposição pode reduzir a complexidade da representação do problema e permitir que diferentes aspectos sejam otimizados de maneira cooperativa. Entretanto, a simplificação adotada pelos autores, considerando profissionais com habilidades equivalentes e atuação restrita a uma única equipe, limita sua aplicação direta a cenários organizacionais nos quais existem diferenças significativas de competências, custos e disponibilidade entre os profissionais.

A incorporação de informações sobre habilidades técnicas e relações de colaboração constitui outro avanço relevante identificado na literatura. A utilização de históricos de projetos e grafos de colaboração permite que a composição das equipes considere não apenas a competência individual, mas também a experiência de colaboração entre os profissionais [4]. Essa característica aproxima o modelo das condições observadas em organizações reais. Entretanto, sua aplicação depende da existência e da qualidade de dados históricos suficientemente representativos, além da definição de mecanismos confiáveis para transformar esses dados em parâmetros utilizados pela função de avaliação.

De maneira complementar, a utilização de um Modelo de Tarefa Estruturada (STM) demonstra que parte da dificuldade do problema não está necessariamente no algoritmo de otimização, mas na própria representação das tarefas e das competências [3]. A utilização de etiquetas tecnológicas e categorias padronizadas permite transformar informações originalmente descritas de maneira pouco estruturada em dados que podem ser processados pelo algoritmo. Essa etapa é importante porque uma solução de otimização somente será tão adequada quanto a representação dos dados utilizada para construí-la.

A partir dessas abordagens, observa-se que uma solução mais abrangente para o problema proposto deveria considerar, de forma integrada, pelo menos quatro dimensões: **competência dos profissionais, disponibilidade e capacidade, características das tarefas e objetivos organizacionais**. Nesse sentido, uma possível arquitetura de solução poderia utilizar um algoritmo evolutivo multiobjetivo para gerar diferentes configurações de alocação, enquanto uma representação estruturada das tarefas forneceria os dados necessários para avaliar a aderência entre profissionais e atividades. Informações históricas de colaboração poderiam

ainda complementar a avaliação das equipes, principalmente em cenários nos quais a interação entre os membros possui impacto relevante sobre o desempenho.

A função objetivo também merece atenção especial. A utilização de um único critério, como a minimização do tempo total do projeto, pode produzir soluções inadequadas do ponto de vista gerencial. Uma equipe formada exclusivamente para minimizar o prazo, por exemplo, pode utilizar profissionais mais caros ou concentrar competências críticas em poucos indivíduos. Da mesma forma, a minimização do custo pode resultar em maior duração ou menor aderência entre competências e tarefas. Dessa forma, a formulação multiobjetivo mostra-se particularmente apropriada para representar os conflitos existentes entre custo, prazo, competência e utilização da capacidade.

Outro aspecto relevante é a distinção entre alocação estática e dinâmica. As abordagens analisadas concentram-se predominantemente na definição de uma configuração de alocação para um determinado cenário. Entretanto, em ambientes reais, a disponibilidade dos profissionais, as prioridades dos projetos e a duração das tarefas podem sofrer alterações durante a execução. Uma abordagem dinâmica poderia utilizar a solução existente como ponto de partida e realizar novas otimizações sempre que ocorrerem eventos relevantes, como indisponibilidade de um profissional, atraso de uma tarefa, alteração de prioridade ou inclusão de uma nova demanda. Essa característica representa uma possibilidade de extensão importante para trabalhos futuros.

Apesar das vantagens das metaheurísticas, sua utilização também apresenta limitações. A qualidade das soluções depende da representação adotada, da definição das funções objetivo, dos parâmetros do algoritmo e dos mecanismos utilizados para tratar soluções inviáveis. Além disso, uma solução matematicamente eficiente não necessariamente será a mais adequada do ponto de vista organizacional, pois fatores como conhecimento tácito, experiência, comunicação, motivação e contexto das equipes podem ser difíceis de representar quantitativamente. Portanto, os resultados produzidos por uma metaheurística devem ser entendidos como suporte à decisão, e não como substituição completa da avaliação do gestor de projetos.

Assim, a questão proposta neste trabalho não deve ser interpretada como a busca por um único algoritmo universalmente superior, mas como a identificação de uma estratégia de otimização compatível com as características do problema. As evidências analisadas indicam que algoritmos genéticos, especialmente em formulações multiobjetivo, apresentam flexibilidade para representar diferentes critérios e restrições, enquanto estratégias cooperativas, modelos estruturados de tarefas e informações históricas de colaboração podem complementar a representação e a avaliação das soluções.

REFERÊNCIAS

- [1] J. Ren, et al, "Cooperative Co-evolutionary Optimization of Software Project Staff Assignments and Job Scheduling", SSBSE 2011,

- [2] C. Stylianoua, et al, "A multi-objective genetic algorithm for intelligent software project scheduling and team staffing", *Intelligent Decision Technologies* 7, DOI 10.3233/IDT-120151, 2013
- [3] A. Costa, et al, "A Genetic Algorithm-Based Approach to Support Forming Multiple Scrum Project Teams", *IEEE Access*, 2022.
- [4] F. Cunha, et al, "A Genetic Algorithm with Convex Combination Crossover for Software Team Formation: Integrating Technical and Collaboration Skills", <https://doi.org/10.1145/3672608.3707980>, SAC '25, 2025.